

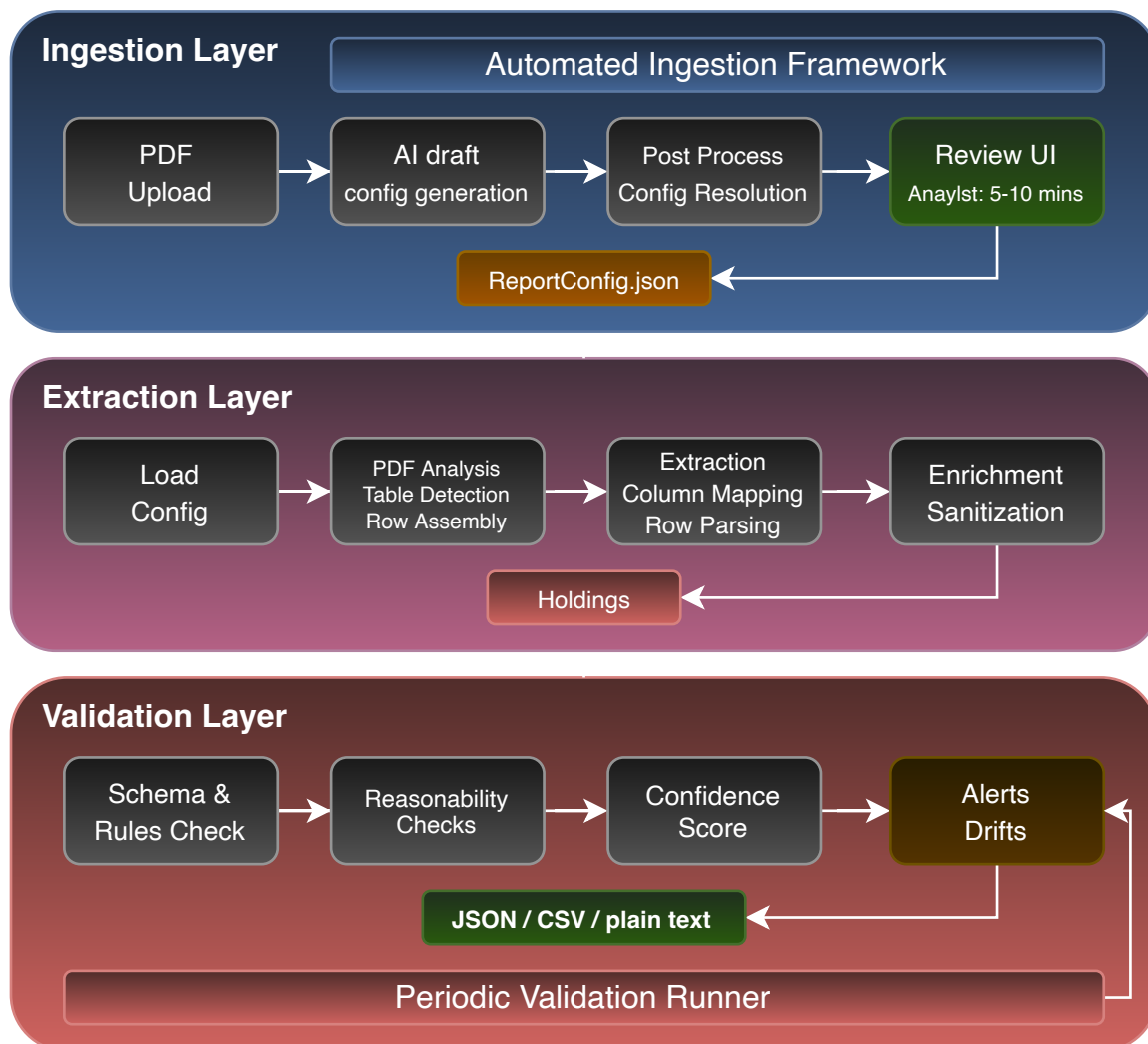
Data Parser Mini Project Challenge

Summary

This document describes the solution design for a system that extracts structured investment holdings data from fund PDF reports. It covers the full pipeline from fund onboarding through automated monthly extraction, including configuration management, and validation.

System Overview

High Level Architecture



Design Summary

- System is structured as a data pipeline and is composed of three layers, **Ingestion**, **Extraction**, and **Validation**
- The generation of a report (or family of reports, e.g. BlackRock, etc.) is driven off a **ReportConfig.json** configuration file

Ingestion

- This layer is responsible for onboarding via manual PDF entry as well as automatic ingestion of pre-configured reports on a quarterly, semi-annually, annual basis, etc.
- For onboarding a fund family, the report PDF is sent to an AI endpoint with a highly detailed prompt, the call returns a draft report config file (further details later in the document)
- A post process resolution step is performed on the draft, resolving column bounding boxes and preparing the config for review
- Analyst has the opportunity to review the proposed config via a visualization UI, with the ability to tweak any config values such as fund names, bounding boxes, table markers, sector whitelists, report groupings, etc.
- UI would be used to update and revise config files in case of future issues
- Config saved for future use: end result: a single AI call + review process onboards the entire fund family resulting in minimal cost

Extraction

- This layer uses existing config files to generate structured data for a given reporting PDF corresponding to the config file fund family
- Any holdings generated from the same config file and PDF will always produce the same output, i.e., this layer is fully deterministic (barring any changes to the code)
- Under the hood, uses a PDF text extraction library to read and analyze files
- Uses bounding boxes and heuristics to dynamically determine the presence of investment tables, column headers, and data ranges, based on data specified in the report config file
- Relies heavily on regex patterns to sanitize and clean data
- Emits holdings data in any format required (JSON, XML, CSV, plaintext, etc.)

Validation

- This layer is extremely critical for the long term viability of the system
- Rules based, custom validation logic supporting hundreds or thousands of different checks
- Runs after the extraction phase to determine any major issues (security names, subtotal, totals, report date, country codes, currencies, etc.)
- A continuous integration framework runs checks continuously against a known set of good outputs to proactively detect any regressions - to be run daily, weekly, monthly, etc., against new reports or whenever changes are made to the ingestion or extraction code

Ingestion

- This layer is the only part of the system that contains non-deterministic elements (draft report config produce by AI)
- Key challenges: bounding box resolution for table coordinates, page continuation, handling multi-line column headers and data, inconsistent table formatting (column headers that span multiple data columns, etc.)
- Onboarding time for a fund family possible in 10 - 15 minutes, enabled by AI-assisted config generation and editing of draft config via review UI

Report Config Schema

- Supports multi-column table formats, multi-line headers across multiple funds in a single report
- Example for BlackRock (see repo for full file):

```
{
  "ReportId": "blackrock-international-fund-series",
  "ReportType": "SingleFund",
  "Publisher": "BlackRock",
  "Version": 1,
  "CreatedAt": "2026-06-09T08:15:34.422842Z",
  "ReportLayout": {
    "TableConfig": {
      "ColumnGroups": [
        {
          "Fields": [
            {
              "Field": "SecurityName",
              "HeaderText": "Security",
              "IsHeaderTextVisible": true,
              "Index": 0,
              "LeftX": 17,
              "RightX": 171
            },
            {
              "Field": "Shares",
              "HeaderText": "Shares",
              "IsHeaderTextVisible": true,
              "Index": 1,
              "LeftX": 200,
              "RightX": 230
            },
            {
              "Field": "MarketValue",
              "HeaderText": "Value",
              "IsHeaderTextVisible": true,
```

Usage of AI

- AI called exactly once per report family using a highly detailed and customizable prompt as below
- Cost is typically on the order of cents per file depending on file size, up to a dollar or two for much larger files

AI Prompt Snippet

- Note: full text available in repo

```
You are a financial data extraction configuration expert specializing in fund report PDFs.
You are now analysing a fund report PDF. This may be a mutual fund or ETF share holder report,
private equity (PE) or Venture Capital (VC) quarterly reports, hedge fund, alternate investment fund (AIF),
reports, or any other investment vehicle. The report may be monthly, quarterly, semi-annual, or annual.

Your task is to produce a single ReportConfig JSON object covering ALL funds in this document.

STEP 1 – Identify the report:
- reportId: lowercase hyphen-separated slug identifying this report family across all periods
  (e.g. "acme-global-equity"). Never include dates, quarters, or years – the same reportId
  must apply to January's report and December's report of the same fund/series.
- reportType: "SingleFund" or "MultiFund"
- publisher: the asset manager or management company name

STEP 2 – Identify the shared layout that applies to most or all funds in this report:
- reportDateRegex: C# regex matching the report as-of date line. Must be generic –
  e.g. "As of \\w+ \\d{1,2},?\\s*\\d{4}" not "As of May 31, 2025"
- tableConfig: describes how the Schedule of Investments table (or equivalent –
  Portfolio of Investments, Schedule of Portfolio Holdings, etc.) is laid out across
  the page width. It contains an ordered list of column groups, each covering a horizontal
  slice of the page:

    columnGroups: array of ColumnConfig objects, one per visual column group, ordered left to right.
    Each ColumnConfig has a fields property:

      fields – an ordered array of FieldEntry objects, one per data column, left to right.

    Each FieldEntry has:
      field      – canonical field name (see list below)
      headerText – verbatim header text exactly as it appears in the PDF
                  (e.g. "Mkt Value", "No. of Shares"); null when the column has no visible header
      index      – 0-based left-to-right position of this column within the column group

Canonical field names (field values in FieldEntry):
  SecurityName  – the name or description of the holding
  SecurityType  – type of instrument (equity, bond, ETF, etc.)
  Sector        – GICS sector or equivalent classification
  Country       – country of risk or domicile (often ISO 3-letter code)
  Shares        – number of shares held
  Principal     – par / principal value (bonds)
  PrincipalOrShares – single column that serves as both shares and principal
  Cost          – cost basis or book value of the holding
  MarketValue   – market value / fair value
  Other         – a column that is present in the table but does not match
```

Extraction

- This layer deals with extraction of holdings given a report config file and PDF report
- The output produced from this layer is always deterministic, which is a key trait in ensuring overall system reliability and long term success

Holdings Output

```
holdings : [
{
  "securityName": "Teck Resources Ltd., Class B",
  "securityType": "Common Stocks",
  "sector": null,
  "country": "Canada",
  "shares": 742465,
  "principal": null,
  "marketValue": 25387670,
  "sourcePageNumber": 2,
  "extractionConfidence": 1,
  "countrySource": "FromPdf",
  "extractionWarnings": []
},
{
  "securityName": "Toronto-Dominion Bank (The)",
  "securityType": "Common Stocks",
  "sector": null,
  "country": "Canada",
  "shares": 411203,
  "principal": null,
  "marketValue": 30875781,
  "sourcePageNumber": 2,
  "extractionConfidence": 1,
  "countrySource": "FromPdf",
  "extractionWarnings": []
},
{
  "securityName": "Waste Connections, Inc",
  "securityType": "Common Stocks",
```

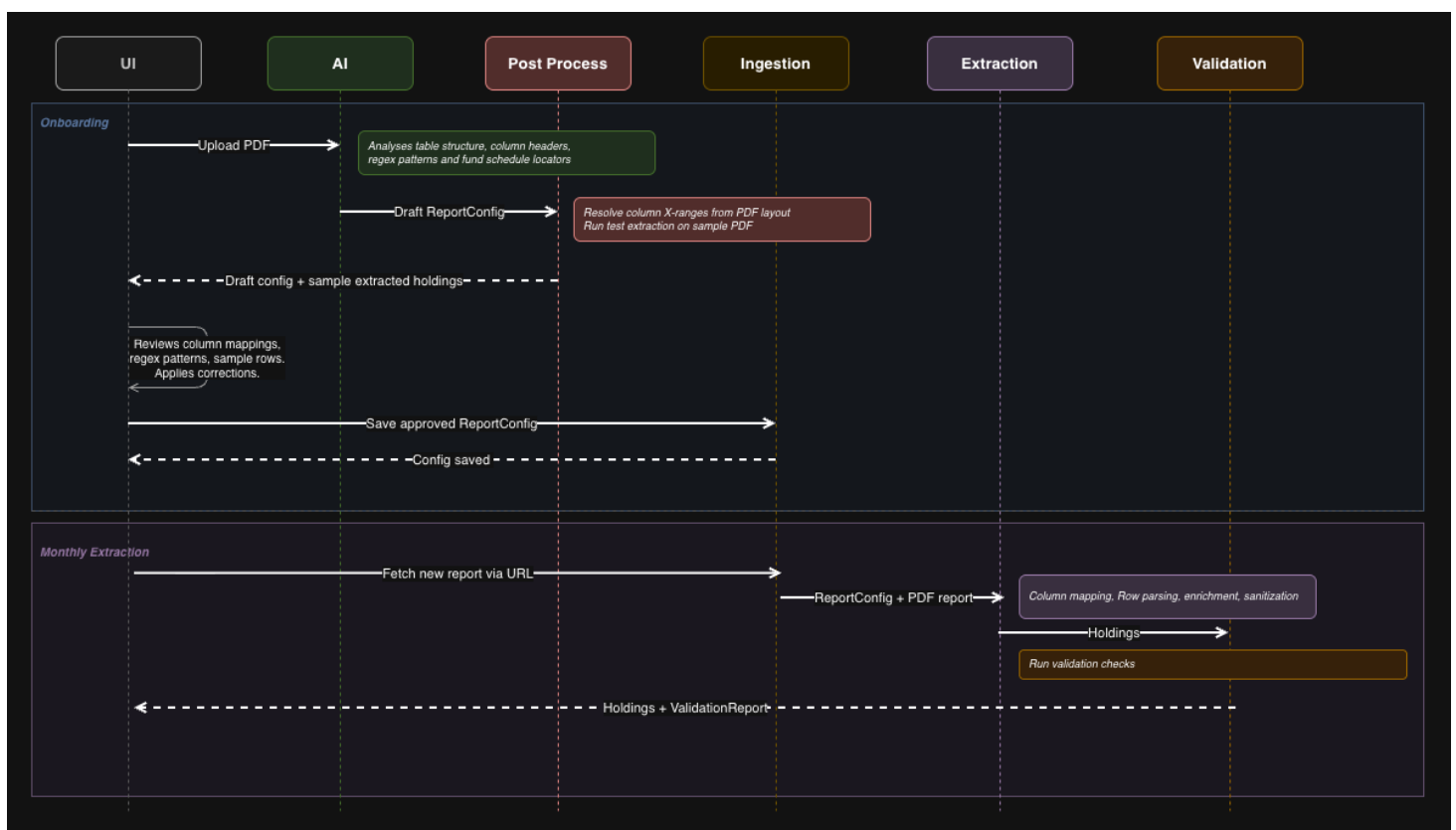
Validation

Sample Validation Failures

```
findings": [
  {
    "ruleName": "CountryCode",
    "message": "'Canada' is not a valid ISO3 country code for holding 'Teck Resources Ltd., Class B'",
    "severity": "Warning",
    "holdingIndex": 0,
    "fieldName": "Country"
  },
  {
    "ruleName": "CountryCode"
```

Workflow

Sequence Diagram



Config Update Workflow

- If a fund layout is updated mid-year, it will be detected by the ingestion framework and updated automatically via the onboarding workflow (barring any major validation failures)
- An analyst can use the review UI to resolve any validation failures as needed

Future Enhancements

- Dynamically changing headers in the same table or multi-table arrangements for the same fund
- Jagged multi-line table data as in the Goldman's report:

Shares	Description	Value
Common Stocks – 98.1%		
China – 93.9%		
99,816	Aier Eye Hospital Group Co. Ltd., Class A (Health Care Equipment & Services)	\$ 176,979
166,076	Alibaba Group Holding Ltd. (Consumer Discretionary)	

- Headers that span multiple columns with misaligned data underneath:

Shares or Principal Amount	Market Value [†]
CORPORATE BONDS - 46.7% - (continued)	
Apparel - 0.1%	
\$ 7,050,000	Tapestry, Inc. 5.50%, 03/11/2035 \$ 6,866,083
Auto Manufacturers - 0.2%	
13,285,000	Daimler Truck Finance North America LLC 5.38%, 01/13/2032 ⁽²⁾ 13,304,065

- Complex multi-level groups (sectors, security type, parent security issuer, etc.)

Scalability

- This design allows for future scalability if needed as it naturally allows for parallel processing of multiple reports at the same time
- Either of the Ingestion, Extraction or Validation layers could be run as micro-services or nano-services and auto-scaled as needed given a burst in demand

Coding Challenges

Column Range Resolution

- Resolving left and right coordinates for a given column to determine how data is parsed for that column (note code is not production-ready):

```
// Capturing several possibilities here.
// Right aligned with column header, left-aligned with column header
// Wider than column header, fits inside column header
// TODO support jutting to the left or jutting to the right
const double alignmentTolerance = 2.5;
var alignedBlocks =
    line.Blocks
        .Where(b => Math.Abs(b.Left - headerBlock.Left) <= alignmentTolerance ||
                    Math.Abs(b.Right - headerBlock.Right) <= alignmentTolerance ||
                    (headerBlock.Left < b.Left && headerBlock.Right > b.Right) ||
                    (b.Left <= headerBlock.Left && b.Right > headerBlock.Right))
        .ToList();
if (alignedBlocks.Any())
{
    candidates.AddRange(alignedBlocks);
}
```

- Resolution of column x-coordinate boundaries when one column does not have a visible header
- Solution was to resolve all visible headers and allocate the remaining space to the non-visible column

```
// Allocate X space to the single non-visible field using its index position within the
// ordered field list to determine which neighbouring resolved ranges bound it.
(double Left, double Right)? nonVisibleRange = null;
if (nonVisibleHeaders.Count == 1 && xRanges.Count > 0)
{
    var nonVisible = nonVisibleHeaders[0];
    var ordered = column.Fields.OrderBy(f => f.Index).ToList();
    var pos = ordered.IndexOf(nonVisible);

    var prev = pos > 0 ? ordered[pos - 1] : null;
    var next = pos < ordered.Count - 1 ? ordered[pos + 1] : null;

    var prevRight = prev is not null && xRanges.TryGetValue(prev.HeaderText!, out var pr) ? pr.Right : (double?)null;
    var nextLeft = next is not null && xRanges.TryGetValue(next.HeaderText!, out var nl) ? nl.Left : (double?)null;

    var uncovered = rows
        .SelectMany(l => l.Blocks)
        .Where(b => !NoisyBlock.IsMatch(b.Text) &&
            (prevRight is null || b.Left > prevRight.Value) &&
            (nextLeft is null || b.Right < nextLeft.Value))
        .ToList();

    if (uncovered.Count > 0)
        nonVisibleRange = (
            Math.Floor(uncovered.Min(b => b.Left)),
            Math.Ceiling(uncovered.Max(b => b.Right)));
}
```

Detecting Multi-line Header Rows

- Column headers can sometimes span multi-lines as with the Vanguard report below:

Shares	Market Value (\$000)
8,664,473	630,514
12,421,406	614,238

- This was solved using a look-back approach, successively concatenating lines above until a match is found (or until we give up at a max line count):

```
// Try to detect multi-line column header
// Look a maximum two lines above the current line
var currHeader = line.Text;
var targetLine = allLines.SingleOrDefault(x => x.Intersect(lines).Any());
if (targetLine is null) continue;
var lineIndex = allLines.IndexOf(targetLine);
var multilineMatch = false;
var innerIndex = 1;

while (innerIndex <= 2 && !multilineMatch)
{
    var prevLine = allLines.ElementAtOrDefault(lineIndex - innerIndex);
    innerIndex++;
    if (prevLine is null) continue;
    foreach (var prev in prevLine)
    {
        var potentialHeader = $"{prev.Text} {currHeader}";
        if (visibleHeaders.Any(h => string.Equals(h, potentialHeader)))
        {
            matched++;
            multilineMatch = true;
            break;
        }
        if (visibleHeaders.Any(h => h.Contains(potentialHeader)))
        {
            currHeader = potentialHeader;
            break;
        }
    }
}
```

Notes

- In order to run this solution locally, you will need the app settings.json file with an Anthropic API key
- More sample config files and holdings output are available the repo in the samples folder
- Currently limited to file size around 5MB due to token limit (can be increased if desired)